LEBANESE INTERNATIONAL UNIVERSITY / CSCI426

WeScore

Project Phase 1 — Report

Football fixtures, teams, standings and favourites • Original responsive React frontend

PROJECT	DETAILS
Course	CSCI426 — Advanced Web Programming
Instructor	Dr. Kawthar Messelmani
Stage	Project Phase 1 — approved football website concept
Stack	ReactJS + React Router + JavaScript + CSS
Data	Fictional frontend sample data; no backend
Academic year	Fall 2026–2027

Prepared as a code-and-method explanation for the accompanying original React source files. The deployed GitHub URL and final student information must be provided by the submitting student.

1. Abstract

WeScore is an approved frontend football-information web application designed as a Phase 1 project for CSCI426 Advanced Web Programming. It addresses the difficulty of discovering match schedules, exploring teams and reviewing league records across separate sources. The application provides seven navigable React pages. Users can browse fictional match fixtures, search and filter results, view club profiles and standings, and save favourite matches or teams in local browser storage. The interface adapts to desktop and mobile screen sizes. Phase 1 intentionally uses demonstration data, not live sports results or a backend. The implementation uses techniques demonstrated in the course lectures, including React components, React Router, useState, array processing, forms, JSON and localStorage.

2. Problem statement and objectives

Football fans frequently use different pages for fixtures, club data and standings. WeScore combines these familiar tasks into one understandable, responsive interface. In this phase, the data is fictional so that search, filter, sort and favourites functionality can be demonstrated without a football-data API.

- Build a functional React frontend with at least five pages and one dynamic page.
- Provide responsive UI that works at both desktop and mobile widths.
- Reuse components and data arrays to keep the code organized.
- Demonstrate local user interaction and saved preferences without a server.
- Document the code, maintain authentic Git history and deploy on GitHub Pages.

3. Requirements fulfillment

Requirement	Implementation
Frontend with ReactJS	React 18 with components, JSX and React Router
5+ pages	Seven pages: Home, Matches, Teams, Standings, Features, About, Contact
Dynamic page	Matches search, filter, sort, and favourites; dynamic Teams and Standings
Responsive design	Flexible grid/flex styling and mobile navigation
Git version control	GITHUB_SETUP.md; requires student to create/push own commits
Deployment	GitHub Pages deployment scripts, requires student's repository
README and screenshots	README.md and docs/screenshots images

4. System design

4.1 Architecture

```
Browser / UI
  |
React entry: src/index.js
  |
App.js — BrowserRouter with 7 page routes
  |-- shared NavBar + Footer
```

```
  |--- Home / About / Features / Contact
  |--- Matches --> MatchesList.js --> MatchCard.js
  |--- Teams   --> TeamsList.js
  |--- Standings --> TeamsList.js
  |
React state + browser localStorage (saved fixtures and teams)
```

4.2 Main data flows

1. The browser loads index.html and index.js renders App.
2. React Router shows the page for the current path while Navbar/Footer persist.
3. Matches and Teams read local array objects and apply state-driven filters.
4. A user clicks a favourite button; React state updates and localStorage stores a JSON string.
5. Contact uses controlled input fields and displays a validation/demo alert; no network request is sent.

4.3 Data model

MatchesList.js contains 12 sample fixtures with fields such as id, home, away, date, time, status, competition, venue and scores. TeamsList.js holds 8 fictional teams with fields such as name, city, played, won, drawn, lost, goalsFor, goalsAgainst and points. All records are explicitly fictional for academic use.

5. Technologies used

Technology	Purpose
ReactJS / JSX	Functional components and reusable rendering
JavaScript	Arrays, objects, filters, sorting and state changes
React Router DOM	Page-to-page navigation
CSS	Layout, design and responsive media queries
localStorage + JSON	Browser persistence for favourites
Git + GitHub	Source control and project submission
GitHub Pages + gh-pages	Static frontend deployment

This design deliberately avoids a backend, database and external football-data API, which are not required by the Phase 1 handout.

6. Key source code snippets

6.1 Routing

```
<Routes>
 <Route path="/" element={<Home />} />
 <Route path="/matches" element={<Matches />} />
 <Route path="/teams" element={<Teams />} />
 <Route path="/contact" element={<Contact />} />
</Routes>
```

6.2 Search and filter

```
const filteredTeams = TeamsList.filter(team =>
 team.name.toLowerCase().includes(search.toLowerCase())
);
```

6.3 Local favourites

```
const updated = favourites.includes(id)
 ? favourites.filter(item => item !== id)
 : [...favourites, id];
setFavourites(updated);
localStorage.setItem('wescore-favourites', JSON.stringify(updated));
```

6.4 Standings sort

```
const sortedTeams = [...TeamsList].sort((a, b) => b.points - a.points);
```

6.5 Contact form

```
const handleChange = (e) => {
 setValues({ ...values, [e.target.name]: e.target.value });
};
const handleSubmit = (e) => {
 e.preventDefault();
 alert(JSON.stringify(values));
};
```

Full original source code is included in the companion repository / ZIP. The Word Code Guide explains these methods and every project file in detail.

7. UI screenshots

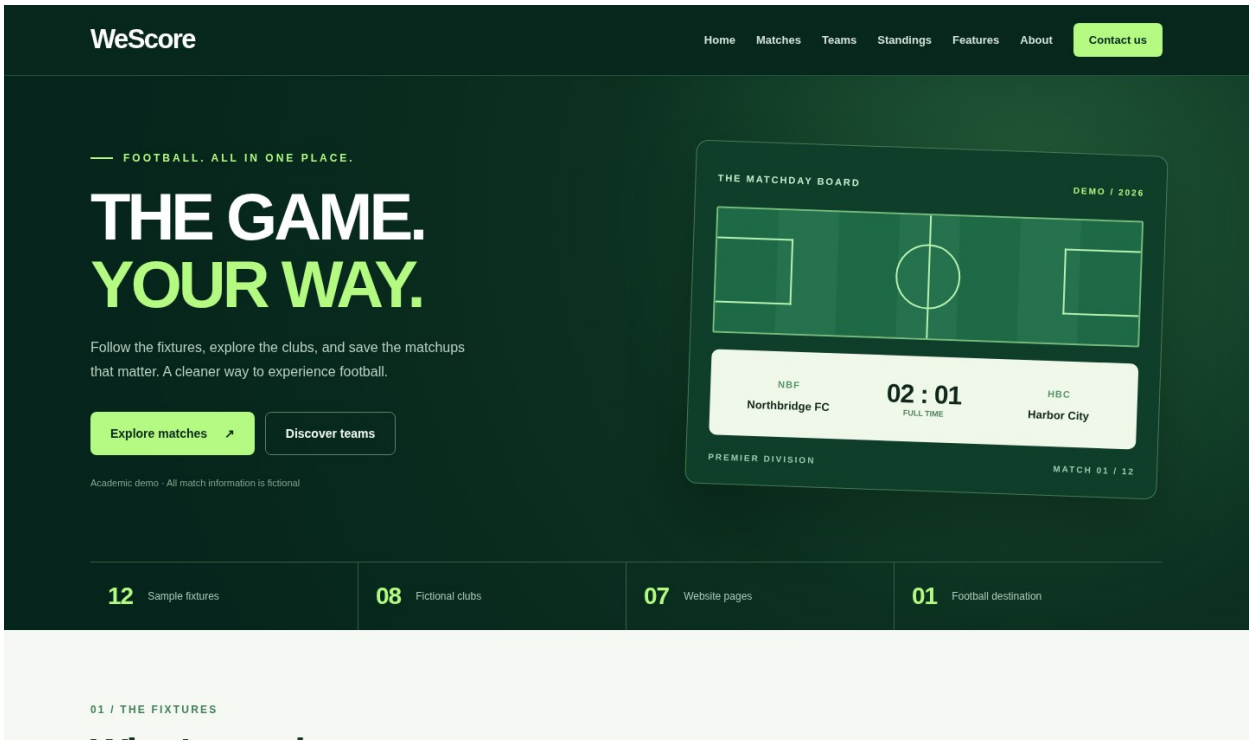


Figure 1 — Home page desktop preview

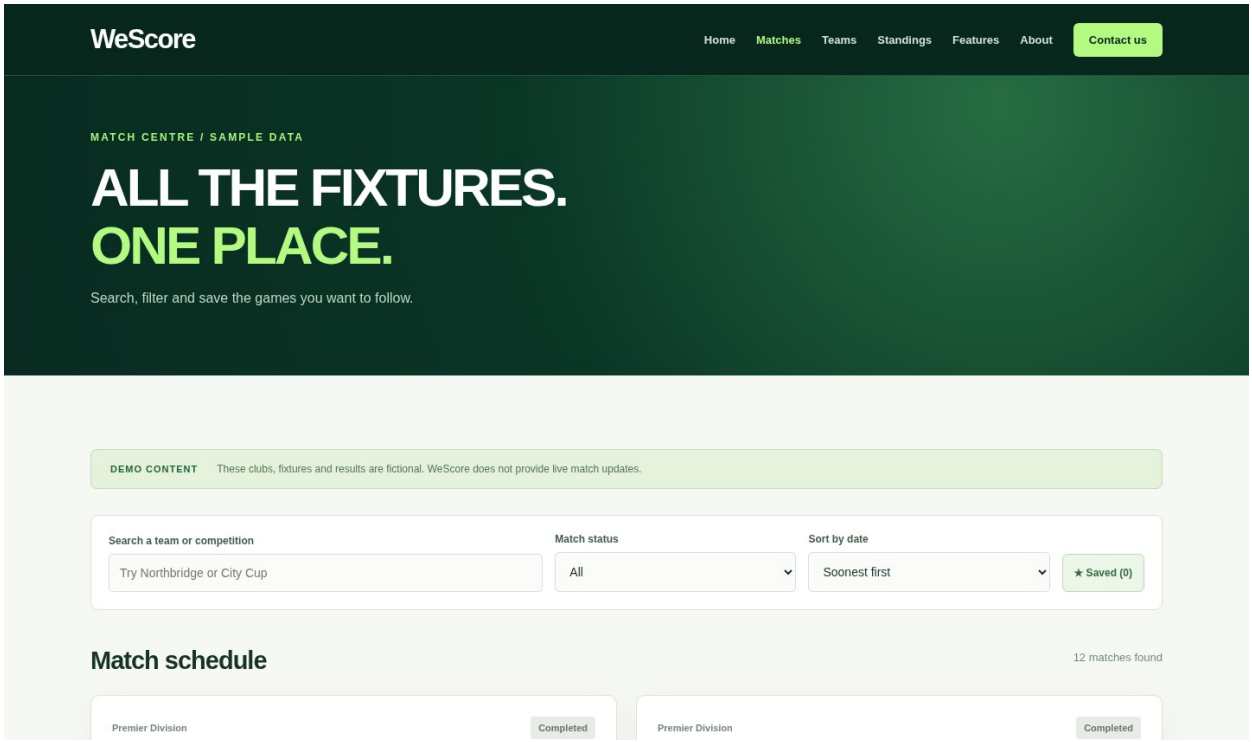


Figure 2 — Matches dynamic page desktop preview

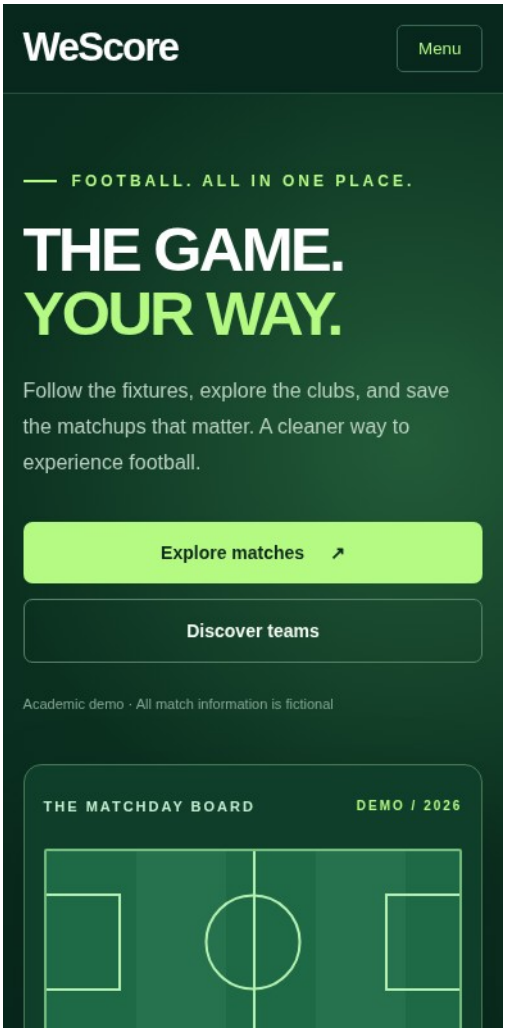


Figure 3 — Home page mobile preview

8. Testing and results

Test	Result
React production build	Compiled successfully
Routes/navigation in local preview	Home, Matches, Teams, Standings, Features, About, Contact rendered
Match list search/filter	City Cup search, completed status filter, 12 fixture cards
Favourite matches	React state and browser-store interaction passed
Team search/favourites	Eight club cards, search and favourites interaction passed
Standings	Eight sorted table rows displayed
Contact form	Valid fields display a demo alert; no email transmission
Mobile layout/navigation	390px viewport screenshot and mobile menu interactions passed

The interactive checks were performed in a local browser preview using the same React components and a preview-only in-memory router because the testing environment restricts browser navigation to local URLs. The deliverable source uses the lecture-taught BrowserRouter. After extracting, the student should perform a final local npm start test and deployment check on their own computer.

9. GitHub version control and deployment plan

The project includes GitHub Pages scripts in package.json, plus detailed GITHUB_SETUP.md instructions. The submitting student must create an empty GitHub repository called WeScore, commit the files, push to origin and set package.json homepage to their username and repository. They can then run npm run deploy and configure GitHub Pages to publish the gh-pages branch. Genuine commit history must be demonstrated. No GitHub remote or live deployment is claimed by this delivered package.

```
git init
git add .
git commit -m "Initial WeScore Phase 1 frontend"
git branch -M main
git remote add origin https://github.com/YOUR-USERNAME/WeScore.git
git push -u origin main

# After setting package.json homepage:
npm run deploy
```

10. Limitations and future scope

- Match and team records are fictional; real scores are not updated.
- The form does not send a message to a server or database.
- localStorage favourites are local to the current browser and can be cleared.
- As a static host, GitHub Pages may 404 on a direct refresh of nested BrowserRouter routes.
- Future phases could introduce a backend and verified live-data API if later permitted by the course.

11. Conclusion

WeScore provides an original and responsive football frontend that satisfies the core functional page and dynamic-interaction requirements of CSCI426 Phase 1, using taught React and JavaScript techniques. Deployment, authentic Git commit history, student names and final submission remain the responsibilities of the submitting student.

12. References / source guidance

Course materials provided by Dr. Kawthar Messelmani: CSCI426 Weeks 1–5 lecture slides, Lecture 6-1, React Router, React Forms, React with Bootstrap FullProject, JSON, React LocalStorage, React Favourite Items App, and React Project Deployment to GitHub Pages. Project specification: CSCI426 Project Phase 1 Fall 2026–2027.

Group contribution statement: to be completed accurately by the student(s) at submission. If submitted individually, identify the sole contributor; if a group of two, specify each person’s actual work.